4. CSS3

En este tema vamos a estudiar, las nuevas posibilidades que nos ofrece CSS3 para crear transformaciones y transiciones en los elementos HTML, pero antes veremos que opciones tenemos para posicionar un elemento en nuestra página.

Posicionamiento de los elementos

Las propiedades de posicionamiento CSS nos permiten colocar un elemento en el lugar de la página que más nos interese.

Los elementos pueden ser colocados utilizando las propiedades `top`, `bottom`, `left` y `right`. Sin embargo, estas propiedades no funcionarán a menos que la propiedad `position` se establezca en primer lugar. En función del método de posicionamiento que utilicemos, estas propiedades funcionarán de una manera u otra. Disponemos de cuatro métodos de posicionamiento diferentes variando el valor de la propiedad `position`:

- `static` (posicionamiento estático): Este es el valor por defecto de la propiedad. Los elementos estáticos se encuentran posicionados de acuerdo al flujo normal de la página, es decir, a continuación del anterior elemento con posicionamiento estático. Los elementos con este posicionamiento no se ven afectados por las propiedades `top`, `bottom`, `left` y `right`.
- `fixed`: Un elemento con la posición fija se posiciona relativo a la ventana del navegador, por lo tanto, no se moverá aunque se haga scroll en la página. Además, estos elementos se eliminan del flujo normal del documento y el resto de elementos se comportarán como si estos elementos no existieran. Esto hace que los elementos posicionados fijos pueden superponerse a otros elementos o entre sí.

  ```
  footer { position:fixed; bottom:0px; left:0px; }
  ```

  En el ejemplo, se especifica un posicionamiento fijo del `<footer>`, y estará colocado siempre en la parte inferior de la pantalla y en el extremo izquierdo. Así, aunque hagamos más o menos alta la ventana del navegador o exista más o menos contenido, el `<footer>` siempre estará fijo en la parte inferior de la misma.
- `relative`: La posición de un elemento con posicionamiento relativo se encuentra relativa a la posición que debería ocupar dentro del flujo normal

del documento. Así, su contenido puede ser movido y/o superpuesto a otros elementos, pero el espacio reservado para el elemento se mantendrá reservado dentro del flujo normal del documento. A menudo, se utilizan elementos con posicionamiento relativo para contener otros elementos con posicionamiento absoluto.

`absolute`: En elemento con posicionamiento absoluto se posiciona de forma relativa al primer elemento padre que tiene un posicionamiento distinto de estático, normalmente relativo y, en ocasiones, absoluto o fijo. Si no se encuentra dentro de ningún elemento con estos posicionamientos, se colocará relativo al elemento `<html>`. En este caso, será muy probable que cuando movamos horizontalmente la ventana del navegador o -todavía más grave- que según tenga el usuario una anchura u otra del navegador el elemento este posicionado en un lugar u otro horizontalmente. Para evitar este grave problema, o bien, introducimos el elemento con posición absoluta dentro de otro elemento con posición relativa, o bien, le damos una posición relativa al body, que es el elemento que contiene al resto de elementos.

Evidentemente, los elementos con posicionamiento absoluto se eliminarán del flujo normal del documento, por lo que el resto de elementos se posicionarán como si estos elementos no existieran. Esto hará que, muchas veces, los elementos con posicionamiento absoluto se superpongan sobre otros elementos.

Elementos superpuestos

Cuando los elementos se posicionan fuera del flujo normal del documento, puede ser que se superpongan. En estas circunstancias, debemos indicar de alguna forma qué elementos se colocan delante y cuáles se colocan detrás. Para esta tarea disponemos de la propiedad `z-index`.

La propiedad `z-index` indica el orden de apilamiento de un elemento (qué elemento debe situarse delante o detrás del resto). Un elemento con un orden de apilamiento mayor está siempre delante de un elemento con un orden menor. Si dos elementos se solapan sin tener especificada la propiedad `z-index`, el que haya sido insertado más tarde en el código html estará sobre el otro.

Veamos el siguiente ejemplo:

```
<header>
  <div id="chincheta1"></div>
  <div id="chincheta2"></div>
  <h1>Título de la página</h1>
</header>
<style>
header { position: relative; padding: 20px 100px; }
#chincheta1 {
  position:absolute;
  left:320px;
  top:-5px;
  width: 35px;
  height: 49px;
  background-image: url("imgs/chincheta.png");
  z-index: 2;
  border: 1px solid black;
}
#chincheta2 {
  position:absolute;
  left:350px;
  top:-5px;
  width: 35px;
  height: 49px;
  background-image: url("imgs/chincheta.png");
  z-index: 1;
}
</style>
```



Título de la página

En el ejemplo tenemos un elemento `<header>` que contiene dos `<div>` vacíos y un `<h1>`. Si nos fijamos en los estilos, el `<header>` tiene posicionamiento relativo, pero como no hemos indicado las propiedades `top`, `bottom`, `left` ni `right`, estas tomarán los valores por defecto, es decir `0px`, por lo tanto, el elemento se colocará como si tuviera posicionamiento estático.

La ventaja de tener el `<header>` con posicionamiento relativo es que al colocar los dos `<div>` con posicionamiento absoluto, su posición será relativa a la posición del elemento `<header>`, ya que este será el primer elemento padre que los contiene con un posicionamiento distinto de estático.

Además, como se aprecia en la imagen, los dos elementos con posicionamiento absoluto están superpuestos, el primero (que tiene borde) tiene la propiedad `z-index` a 2, mientras que el segundo (que no tiene borde) tiene la propiedad a 1, por lo tanto, el primero estará colocado sobre el segundo.

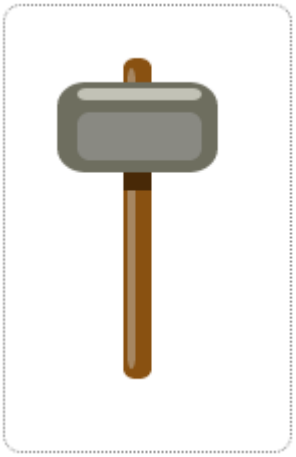
Transformaciones (`transform`)

Con HTML5 y CSS3 podemos utilizar la propiedad `transform` para modificar características de un elemento como la posición (`translate`), la rotación (`rotation`), la escala (`scale`) o el sesgo (`skew`).

Este cambio de posición, tamaño o rotación se produce al instante (no hay animación en este cambio, para ello habría que utilizar otras propiedades que veremos más adelante).

Para poder comprobar estas propiedades vamos a utilizar un `<div>` que contiene el dibujo sencillo de un martillo creado con SVG[1].

```
<div id="martillo1">
 <svg width="250" height="250" xmlns="http://www.w3.org/2000/svg">
  <!-- Mango (marrón oscuro) -->
  <rect x="43" y="10" width="14" height="160" rx="6" ry="5" fill="#895111"/>
  <!-- Brillo mango (marrón claro) -->
  <rect x="45" y="15" width="4" height="150" rx="6" ry="6" fill="#A68661"/>
  <!-- Cabeza hierro -->
  <rect x="10" y="22" width="80" height="45" rx="14" ry="12" fill="#6E6E5F"/>
  <!-- Brillo cabeza -->
  <rect x="20" y="37" width="62" height="24" rx="6" ry="6" fill="#898982"/>
  <rect x="20" y="25" width="62" height="6" rx="4" ry="4" fill="#C2C2B6"/>
  <!-- Sombra de la cabeza en el mango -->
  <rect x="43" y="67" width="14" height="9" fill="#482907"/>
 </svg>
</div>
#martillo
{
  position:relative;
  width:110px;
  height:190px;
  padding:16px;
  border:1px dotted gray;
  border-radius:9px;
  float:left;
}
```



A este `<div>` le vamos a aplicar todas las transformaciones de posición, rotación, escala y sesgo, partiendo de esta posición, tamaño, escala y apariencia original.

Movimiento (translate)

El movimiento se basa en la suma o resta de píxeles en su posición horizontal y vertical respecto a la posición en la que se encuentra el objeto en este momento.

Con números positivos se desplaza el objeto hacia la derecha y hacia abajo y con números negativos se desplaza hacia arriba y hacia la izquierda. Así, en este ejemplo, el martillo se desplaza 200 píxeles hacia la derecha y 5 píxeles hacia abajo respecto a su posición original.

```
#martillo { transform:translate(200px,5px); }
```

También podemos utilizar funciones independientes para cada dimensión: `translateX` y `translateY`.

[1] Los Gráficos Vectoriales Redimensionables (del inglés Scalable Vector Graphics) o SVG son una especificación para describir gráficos vectoriales bidimensionales en formato XML.

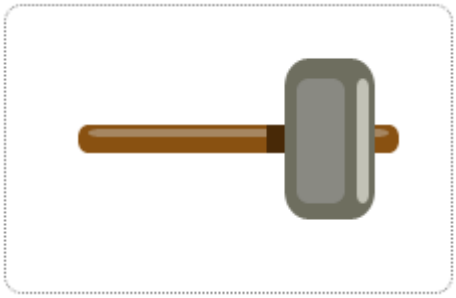
También es posible utilizar porcentajes, si queremos colocar este martillo en la mitad horizontal de la pantalla, es decir, centrar el martillo horizontalmente, podemos indicar un porcentaje (50%), en lugar de utilizar píxeles.

```
#martillo { transform:translate(50%,0px); }
```

Rotación (`rotate`)

La rotación se basa en los grados de rotación (de 0º a 360º). El eje de rotación siempre está situado en el centro del elemento y los grados se especifican añadiendo `deg` después del valor (por ejemplo: `90deg`).

Si se indican valores positivos en los grados de rotación, el objeto rota hacia la derecha. Con los grados especificados en números negativos, el objeto rota hacia la izquierda. (Así, teniendo en cuenta que una vuelta entera equivale a 360º, se aprecia el mismo resultado con `350deg`, que con `-10deg`).



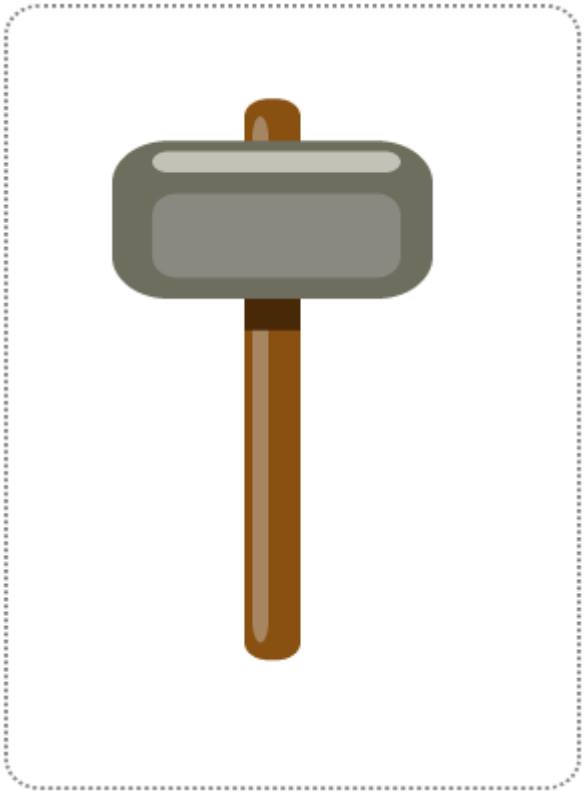
```
#martillo{ transform:rotate(90deg); }
```

En este caso, el elemento del martillo rota 90º hacia la derecha tomando como eje de rotación el centro del `<div>` donde se encuentra el martillo (que no siempre coincide con el contenido de dicho `<div>`).

Escala (`scale`)

Esta propiedad se basa en una escala horizontal, seguida de otra vertical, siendo 1 la escala original. Para obtener un porcentaje se puede multiplicar el valor indicado por 100, así un valor de '2' (2x100=200) representaría un 200%, es decir, el doble de tamaño.

El elemento aumenta o disminuye de tamaño tomando como eje su centro.



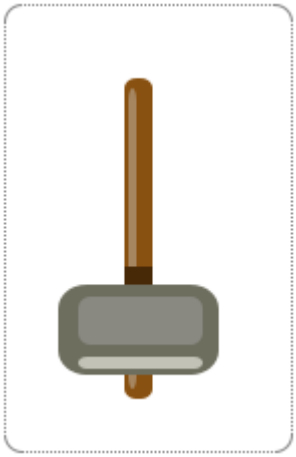
```
#martillo {
transform:scale(2, 1.75);
}
```

En este caso, el elemento del martillo

no aumenta de tamaño de forma proporcional. Aumenta el doble de anchura (2x100= 200%) y sólo un 175% de altura (1.75x100= 175%).

Podemos conseguir el efecto del espejo utilizando números negativos para la escala.

```
#martillo { transform:scale(1, -1); }
```



En el ejemplo, se mantiene la escala en el eje `x`, ya que estamos utilizando el valor 1 que mantiene la proporción original para la dimensión horizontal. Sin embargo, el valor -1 en el eje `y`, aunque también mantiene la proporción original en esta dimensión, invierte el elemento verticalmente para producir el efecto espejo, como se puede apreciar en la imagen.

Existen también otras dos funciones similares a `scale`, pero restringidas a la dimensión horizontal o vertical: `scaleX` y `scaleY`. Estas funciones, por supuesto, utilizan un único parámetro.

Sesgo (`skew`)



Podemos entender el sesgo como la rotación del elemento en los ejes x e y. Para definirlo se indica primero la rotación en el eje x en grados, y posteriormente, la rotación en el eje y, también en grados.

Se pueden utilizar valores positivos o negativos, según se requiera para la deformación.

```
#martillo { transform:skew(10deg, 25deg); }
```

Si sólo indicamos el primer parámetro, únicamente se rotará el elemento sobre el eje `x`. Al igual que ocurría con la función `rotate` podemos utilizar funciones diferentes para cada dimensión: `skewX` y `skewY`.

Múltiples transformaciones simultáneas



Se pueden indicar, de manera simultánea, tantas transformaciones como se deseen, separando cada una de ellas por un espacio.

```
#martillo { transform:translate(200px, 0px) rotate(33deg)
scale(0.5,0.5); }
```

En este ejemplo, al `<div>` del martillo se le aplica un

desplazamiento hacia la derecha de 200 píxeles, seguidamente una rotación hacia la derecha de 33 grados y, finalmente, una reducción de tamaño proporcional del 50%.

Compatibilidad con todos los navegadores

Es importante tener en cuenta que para que `transform` funcione en todos los navegadores deberemos repetir la propiedad anteponiendo los prefijos de cada uno de los navegadores:

- `-webkit-` (para Chrome y Safari)
- `-moz-` (para Firefox)
- `-o-` (para Opera)
- `-ms-` (para Internet Explorer)

```
#martillo{
   transform:translate(200px, 0px) rotate(33deg) scale(0.5,0.5);
   -webkit-transform:translate(200px, 0px) rotate(33deg) scale(0.5,0.5);
   -moz-transform:translate(200px, 0px) rotate(33deg) scale(0.5,0.5);
   -o-transform:translate(200px, 0px) rotate(33deg) scale(0.5,0.5);
   -ms-transform:translate(200px, 0px) rotate(33deg) scale(0.5,0.5);
}
```

Veamos la tabla de compatibilidad que nos ofrece `Can I use?` para esta propiedad (URL: http://caniuse.com/#feat=transforms2d):



Transformaciones dinámicas

Las transformaciones vistas hasta ahora cambiarán la forma de la web, pero la mantendrán tan estática como siempre. Sin embargo, podemos aprovecharnos de la combinación de transformaciones y pseudo-clases para convertir nuestra página en una aplicación dinámica:

```
#martillo
{
  position:relative;
  width:110px;
  height:190px;
  padding:16px;
  border:1px dotted gray;
  border-radius:9px;
  float:left;
}
#martillo:hover { transform: scale(1,-1); }
```

En este ejemplo, la regla original del elemento `martillo` se ha conservado intacta, pero hemos añadido una nueva regla para aplicar efectos de transformación usando la pseudo-clase `:hover`. El resultado obtenido es que cada vez que el puntero del ratón pase sobre este elemento, la propiedad `transform` lo invertirá sobre el eje vertical, y cuando el puntero salga del elemento, volverá a rotar para volver a su posición original.

Esta técnica ya se utilizaba con versiones anteriores de CSS para crear rollovers, por ejemplo, pero con la nueva propiedad `transform` se pueden conseguir efectos más logrados.

Transiciones (`transition`)

Las transiciones (`transition`) funcionan como pequeñas animaciones que siguen las mismas características que las transformaciones (cambio de posición, tamaño, rotación y/o muchas otras propiedades CSS), aunque con la capacidad de poder especificar un tiempo para que se produzca dicha transformación, y así poder utilizarla como una animación sencilla.

Aunque no es obligatorio, tenemos la posibilidad de especificar el tipo de movimiento y si debe existir algún retardo para que empiece la animación. Además, es interesante poder especificar todos estos parámetros en una sola línea.

```
transition:[propiedad a modificar] [Duración] [Tipo de animación] [Retardo];
transition: width 2s ease-out 0.5s;
```

- [Propiedades a modificar]: Las propiedades que podemos modificar utilizando transiciones son las que se muestran en la siguiente lista:

`all`	`bottom`	`margin`	`text-shadow`
`background-color`	`left`	`opacity`	`vertical-align`
`border`	`right`	`word-spacing`	`visibility`
`border-radius`	`box-shadow`	`letter-spacing`	`z-index`
`color`	`width`	`fill`	
`top`	`height`	`padding`	
	`line-height`	`stroke`	

- [Duración en segundos]: Se debe especificar el número de segundos que va a durar la animación. Primero se escribe un valor numérico (los segundos) y seguido la "s" (ejemplo: 3s).
- [Tipo de animación]: Esta propiedad es opcional. Aquí tenemos la posibilidad de indicar un tipo de `easing`, es decir, si la animación empezará suavemente e irá acelerándose progresivamente, si pasará todo lo contrario o si el movimiento tendrá una velocidad uniforme durante todo su recorrido entre otras. Algunas de las posibilidades son las siguientes:
  - `linear`: La velocidad de la animación es uniforme en todo el recorrido.
  - `ease`: La velocidad se acelera al inicio, luego se retarda un poco y se acelera al final de nuevo.
  - `ease-in`: La animación empieza lentamente y va aumentando progresivamente.
  - `ease-out`: La animación empieza muy rápida y va descendiendo progresivamente.
  - `ease-in-out`: La animación empieza y acaba lentamente, y es en la parte central del recorrido donde la velocidad es más rápida.



- [Retardo]: En este apartado podemos especificar un tiempo (en segundos) que el navegador esperará antes de poner en marcha la animación. Se especifica el número de segundos a esperar seguido de la "s" (ejemplo: 1s).

Compatibilidad con todos los navegadores

Al igual que ocurría con las transformaciones, para que `transition` funcione en todos los navegadores deberemos repetir la propiedad anteponiendo los prefijos de cada uno de los navegadores:

- `-webkit-` (para Chrome y Safari)
- `-moz-` (para Firefox)
- `-o-` (para Opera)
- `-ms-` (para Internet Explorer)

Hasta hace poco Internet Explorer 9 todavía no era compatible con las transiciones, mientras que el resto de navegadores lo aceptan desde versiones anteriores a las actuales.

Veamos la tabla de compatibilidad que nos ofrece `Can I use?` para esta propiedad (URL: http://caniuse.com/#feat=css-transitions):



Transición con una única propiedad

Vamos a hacer que el elemento martillo con el que hemos trabajado anteriormente, cambie su color de fondo (`background-color`) utilizando para ello una transición. La transición tendrá una duración de 1 segundo, el tipo de animación será `ease-in-out` y tendrá un retardo de medio segundo. En primer lugar, indicaremos en la regla css del martillo como se realizará la transición de la propiedad `background-color` si esta fuera cambiada.

```
#martillo
{
  position:relative;
  width:110px;
  height:190px;
  padding:16px;
  border:1px dotted gray;
  border-radius:9px;
  float:left;
  transition: background-color 1s ease-in-out 0.5s;
}
```

A continuación, utilizaremos la pseudo-clase `:hover` para cambiar el color de fondo del elemento:

```
#martillo:hover { background-color: black; }
```

El efecto obtenido será que al pasar con el ratón sobre el martillo se producirá la transición.

Transición con múltiples propiedades

Para especificar el cambio de varias propiedades con diferentes tiempos para cada una de ellas, únicamente tenemos que separar las diferentes propiedades por "," y, posteriormente, modificarlas en el lugar del código necesario.

Así, en el siguiente ejemplo, al pasar el ratón por encima, se producirán 4 cambios de propiedades al mismo tiempo:

- El color de fondo (`background-color`) del martillo se convertirá en negro en 1 segundo.
- La anchura se modificará hasta llegar a los 225 píxeles en 1 segundo.
- La altura se modificará hasta llegar a los 200 píxeles durante 1 segundo.
- El ancho del borde del martillo (`border-width`) se expandirá hasta llegar a los 10 píxeles en 1 segundo.

```
#martillo {
  transition: background-color 1s, width 1s, height 1s, border-width 1s;
}

#martillo:hover {
  background-color: black;
  width:225px;
  height:200px;
  border-width:10px;
}
```

Transiciones con propiedades CSS3

Debemos tener en cuenta que cuando realizamos transiciones utilizando propiedades css3 que requieren el uso de prefijos, estos debemos indicarlos, tanto en la propiedad `transition` como en la propiedad a modificar.

Por ejemplo, para realizar la transformación del espejo creada anteriormente utilizando una transición, haremos lo siguiente:

```
#martillo
{
  position:relative;
  width:110px;
  height:190px;
  padding:16px;
  border:1px dotted gray;
  border-radius:9px;
  float:left;
  transition: transform 1s ease-in-out 0.5s;
  -moz-transition: -moz-transform 1s ease-in-out 0.5s;
  -webkit-transition: -webkit-transform 1s ease-in-out 0.5s;
  -o-transition: -o-transform 1s ease-in-out 0.5s;
  -ms-transition: -ms-transform 1s ease-in-out 0.5s;
}
#martillo:hover {
  transform: scale(1,-1);
  -moz-transform:scale(1,-1);
  -webkit-transform:scale(1,-1);
  -o-transform:scale(1,-1);
  -ms-transform:scale(1,-1);
}
```

Recordad que si queréis reproducir los ejemplos que se han indicado, debéis añadir las líneas correspondientes para los navegadores que utilicéis.

Un ejemplo práctico. Menús desplegables con transiciones.

Hasta la aparición de CSS3 los menús desplegables se basaban, principalmente, en javascript o flash, pero desde la aparición de las transiciones y otros efectos de esta nueva versión de CSS, podemos crear múltiples efectos sin necesidad de programar nada ni utilizar ningún plug-in externo.

Vamos a ver un ejemplo de menú desplegable utilizando transiciones y algún truco CSS para conseguir un aspecto más atractivo.

En primer lugar, el código HTML que necesitaremos será el siguiente:

```
<nav>
  <ul>
    <li><a href="#">Opción 1</a>
      <ul>
        <li><a href="#">Subopción 1</a></li>
        <li><a href="#">Subopción 2</a></li>
        <li><a href="#">Subopción 3</a></li>
      </ul>
    </li>
    <li><a href="#">Opción 2</a>
      <ul>
        <li><a href="#">Subopción 4</a></li>
```

- Opción 1
  - Subopción 1
  - Subopción 2
  - Subopción 3
- Opción 2
  - Subopción 4
  - Subopción 5
  - Subopción 6
  - Subopción 7

```
        <li><a href="#">Subopción 5</a></li>
        <li><a href="#">Subopción 6</a></li>
        <li><a href="#">Subopción 7</a></li>
      </ul>
    </li>
  </ul>
</nav>
```

Con el código anterior tendremos dos opciones de menú, la primera desplegará un menú con tres subopciones y la segunda desplegará un menú con cuatro subopciones. Antes de aplicarle los estilos, la página se verá como se muestra en la imagen.

A continuación, veremos los estilos que necesitamos.

En primer lugar, inicializaremos los márgenes internos y externos de todos los elementos a 0px mediante el selector universal:

```
* { margin: 0; padding: 0; }
```

Para los enlaces del menú principal bastará con quitar el subrayado y cambiar el color de la fuente cuando el ratón está sobre ellos:

```
nav>ul>li>a { text-decoration:none; }
nav>ul>li>a:hover { color: red; }
```

Para convertir la lista desordenada en un menú utilizaremos los siguientes estilos:

```
nav>ul>li {
  position:relative;
  list-style:none;
  display:inline-block;
  padding: 20px 0px;
  margin: 0px 80px;
}
```

Cabe destacar la propiedad `position: relative;` que utilizamos para poder poner el menú desplegable con posición absoluta. De esta forma, la posición del menú desplegable será relativa a la opción que lo contiene.

Los estilos que utilizaremos para el menú desplegable serán los siguientes:

```
nav li ul {
/* sacamos el submenú del flujo normal de los contenidos poniéndole posición
absoluta */
  position:absolute;

/* Le ponemos un z-index de 2 para que en el caso que el submenú se solape
con otros elementos quede por encima de ellos */
  z-index:2;
  list-style:none;

/* El ancho que le pondremos dependerá de lo que contengan los submenús. Lo
que pongamos en la propiedad margin-left será la mitad de lo que hayamos
puesto aquí */
  width:100px;
  text-align:center;
```

```
/* Para centrar el submenú con respecto a la opción de menú, pondremos su
parte izquierda en la mitad del ancho y le restaremos a los márgenes la mitad
del ancho que hayamos puesto anteriormente */
  left:50%;
  margin-left:-50px;

/* el valor de la propiedad top dependerá de lo que contengan las opciones de
menú principales, pondremos un valor adecuado para que el submenú se muestre
debajo. */
  top:25px;

  font:Verdana;
  font-size: 0.7em;

/* ponemos las propiedades height: 0px y overflow: hidden para que el submenú
quede oculto. La propiedad height será la que utilizaremos para animar la
transición */
  height:0px;
  overflow:hidden;

/* aquí configuramos la transición para cuando se cambie el valor de la
propiedad height. Esto hará que la aparición y desaparición del submenú esté
animada */
  -webkit-transition:height 0.4s linear;
  -moz-transition:height 0.4s linear;
  -o-transition:height 0.4s linear;
  transition:height 0.4s linear;
}
```

El siguiente paso será hacer que cambie la altura del submenú cuando pasamos el ratón sobre los elementos del menú:

```
nav li:hover ul { height:200px; }
```

Como se puede observar, utilizamos la pseudo-clase `:hover` en el elemento `li`, pero las propiedades que cambiamos son las del elemento `ul` que está dentro.

Con esto hemos conseguido que el submenú desaparezca y vuelva a aparecer cuando pasamos el ratón sobre la opción correspondiente, pero el efecto de despliegue no se aprecia demasiado porque no hemos aplicado estilos a las opciones del submenú. A continuación, aplicaremos una serie de estilos que mejorarán el aspecto de nuestro menú desplegable.

```
/* Con esto pondremos un color de fondo para los elementos del menú que sean
pares y otro para los impares. También cambiaremos el color de fondo cuando
pasemos el ratón sobre ellos. */
nav li ul li { background-color: #313131; }
nav li ul li:nth-child(odd) { background-color: #363636; }
nav li ul li:hover { background-color: #444; }

/* con esto redondearemos las esquinas del submenú, además de colocarlo
debajo de la opción de menú correspondiente */
nav li ul li:first-child {
  border-radius:3px 3px 0 0;

/* Lo que pongamos como margen superior, deberá coincidir con lo que hemos
puesto en el top del ul */
```

```
  margin-top:25px;

/* indicamos posición relativa para poder introducir luego el triángulo del
menú desplegable con posición absoluta */
  position:relative;
}

nav li ul li:last-child {
  border-bottom-left-radius:3px;
  border-bottom-right-radius:3px;
}

/* Con esto conseguiremos que aparezca el triángulo
superior que aparece en el menú desplegable */
nav li ul li:first-child:before
{
      content:'';
      position:absolute;
      width:1px;
      height:1px;
      border:5px solid transparent;
      border-bottom-color: #313131;
      left:50%;
      top:-10px;
      margin-left:-5px;
}

/* Aquí configuraremos la visualización de los enlaces del menú desplegable
*/
nav li ul li a
{
      padding:12px;
      color:#fff;
      text-decoration:none;
      display:block;
}
```



Como se puede apreciar en la imagen, la apariencia del menú está bastante lograda y, si lo probamos, veremos que el efecto que se obtiene está bastante conseguido.